MPI HIE Engine

Performance mechanisms: a Java, database, distributed-systems, and event-driven study guide

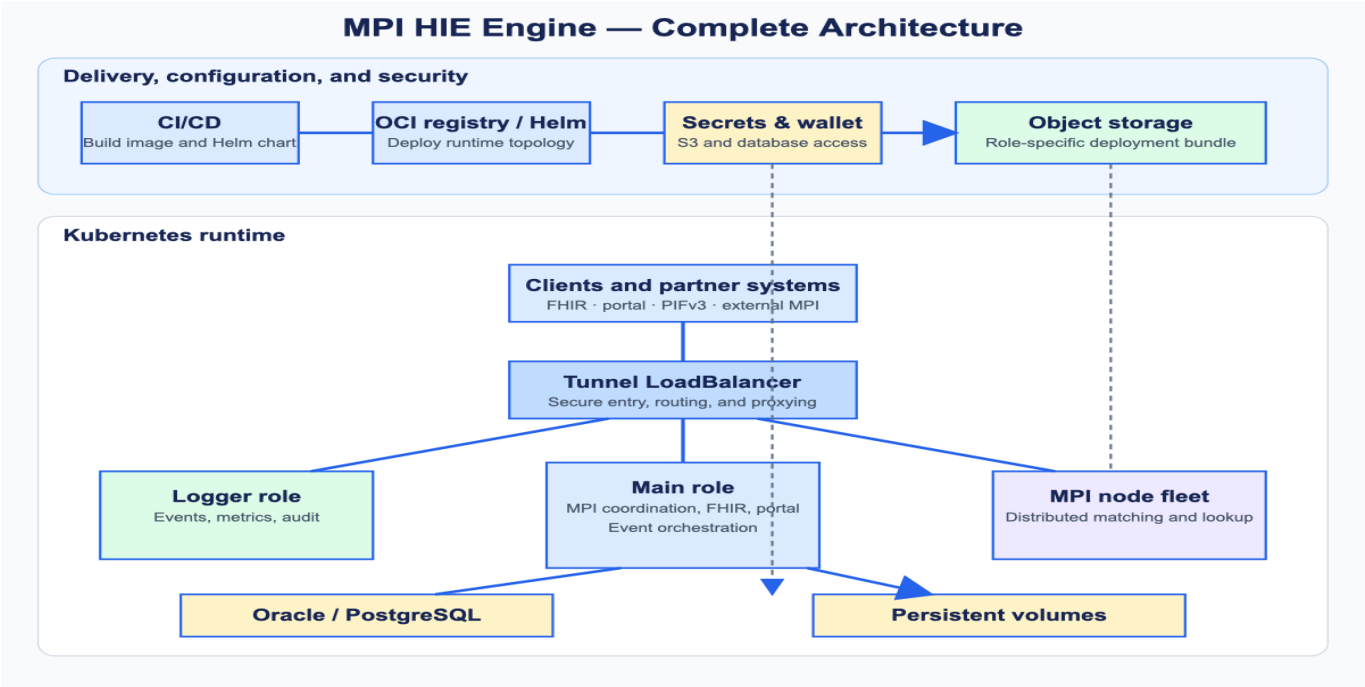
Fast lookup	**Bounded concurrency**	**Efficient persistence**	**Decoupled side effects**

What this guide establishes

The inspected Java runtime is not proven to use reactive streams or Spring WebFlux for its core MPI path. Its database path is blocking JDBC/Hibernate. The performance story is therefore concrete and architectural: in-memory indexes, distributed nodes, thread-pool-backed bulk work, JDBC batching, connection and statement reuse, and asynchronous/event-driven work outside the critical identity path.

How to use this guide

For each topic, learn the idea first, relate it to the evidence in this repository, then prepare the deeper study keywords. Do not claim a latency number without benchmark evidence from the target environment.


MPI HIE Engine — Complete Architecture
Delivery, configuration, and security
CI/CD
Build image and Helm chart
OCI registry / Helm
Deploy runtime topology
Secrets & wallet
S3 and database access
Object storage
Role-specific deployment bundle
Kubernetes runtime
Clients and partner systems
FHIR · portal · PIFv3 · external MPI
Tunnel LoadBalancer
Secure entry, routing, and proxying
Logger role
Events, metrics, audit
Main role
MPI coordination, FHIR, portal
Event orchestration
MPI node fleet
Distributed matching and lookup
Oracle / PostgreSQL
Persistent volumes

1. Fast lookup: in-memory indexes and data locality

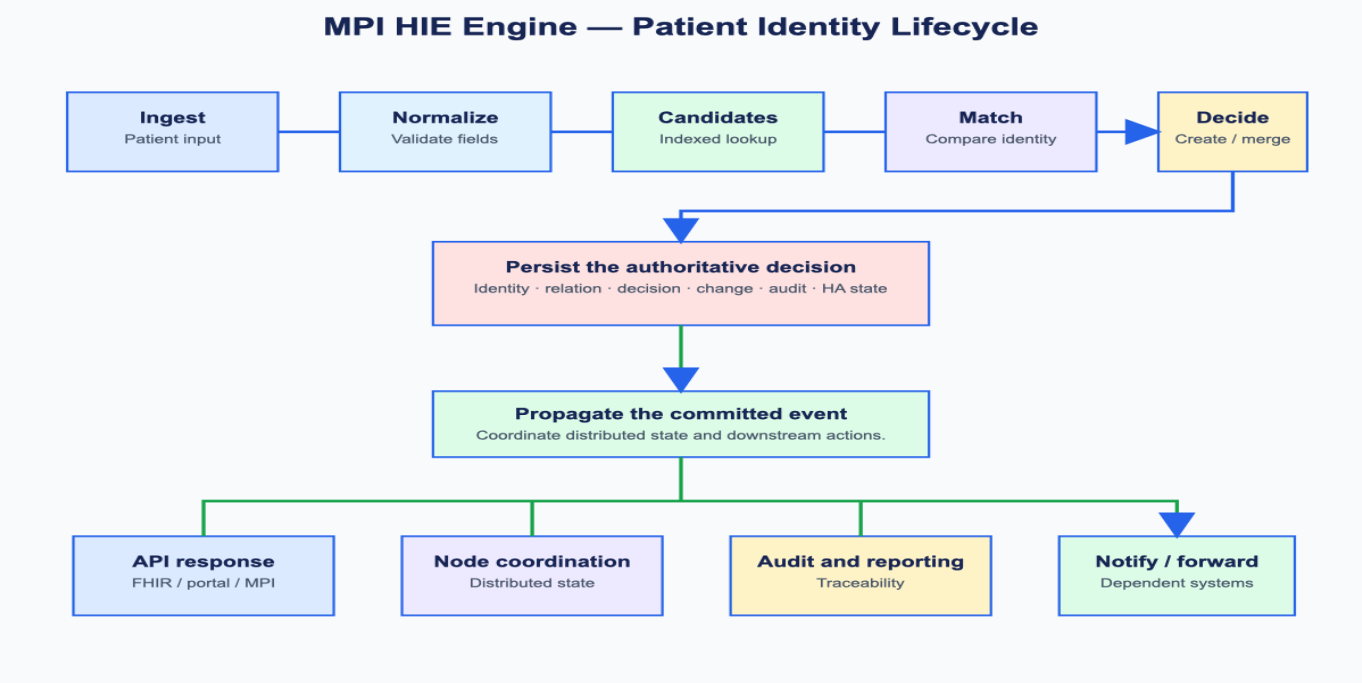
Topic introduction. An index turns a broad search into a targeted candidate lookup. In a patient identity platform, that means reducing the amount of data examined before match rules run.

Why it matters. Matching is often the expensive part of an MPI request. Direct lookup structures reduce repeated database scans and make the candidate set smaller.

Patient ID	Local PID	HIE ID	Field indexes	Candidate set

Internal working in this runtime. IndexedPatientStorage keeps maps for patient ID, local PID, and HIE ID, plus configured patient-field indexes. Node services keep this working set close to the matching logic.

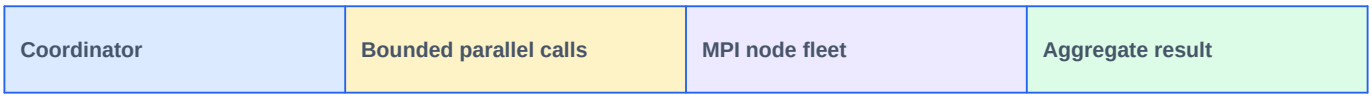
Prepare next. Java HashMap and ConcurrentHashMap, Big-O complexity, inverted indexes, composite indexes, cache invalidation, cardinality, memory sizing, JVM heap and GC.



2. Parallel execution: distributed nodes and bounded concurrency

Topic introduction. Parallelism means doing independent work concurrently while bounding resource usage.

Why it matters. A single JVM has a finite CPU, heap, connection, and network budget. A node fleet increases aggregate capacity; limits protect the system from unbounded fan-out.



Internal working in this runtime. The Helm topology deploys a main coordinator and a fleet of MPI nodes. Main configuration sets parallel limits for calls to nodes and remote interfaces. The Java bulk path contains a ThreadPoolExecutor rather than a reactive event loop.

Prepare next. ThreadPoolExecutor, ExecutorService, queue selection, core versus max threads, CPU-bound versus I/O-bound tasks, Little's Law, backpressure, bulkheads, distributed fan-out/fan-in.

3. Write throughput: buffering, batching, and transactions

Topic introduction. Batching groups several logical operations into fewer database interactions.

Why it matters. Per-row network trips, statement parsing, and commits are expensive. Carefully bounded batches lower overhead while preserving a clear transaction boundary.

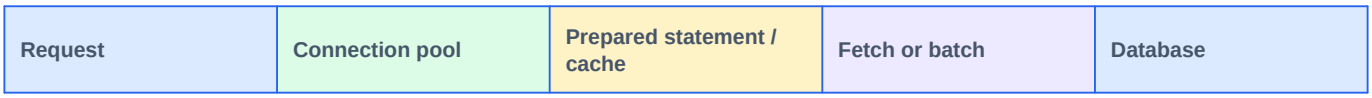
Internal working in this runtime. The profile enables MPI bulk operations with a 1 ms buffer window and maximum size of 100. BulkOperationExecutor owns the buffer and a ThreadPoolExecutor. DBPatientPersistenceStorage reuses PreparedStatement objects and invokes JDBC addBatch for patient writes.

Prepare next. JDBC PreparedStatement, addBatch and executeBatch, batch-size tuning, ACID transactions, autocommit, idempotency, duplicate handling, deadlocks, lock contention, write amplification.

4. Database efficiency: pooling, statement caching, and read tuning

Topic introduction. Efficient database access avoids paying connection and SQL preparation cost repeatedly.

Why it matters. Database connectivity often dominates tail latency. Pooling bounds concurrent demand; prepared statements and statement caches reduce repetitive work; fetch-size tuning makes large reads more predictable.



Internal working in this runtime. The main profile configures Hibernate/C3P0 with a maximum of 300 connections. The persistence code uses prepared statements, adjusts fetch size for loaders, controls transactions, and enables Oracle implicit statement caching when the driver supports it.

Prepare next. Connection pools: C3P0 and HikariCP, pool sizing, database session limits, prepared statements, Oracle statement cache, JDBC fetch size, cursors, explain plans, indexes, query profiling.

5. Event-driven design: protect the response path

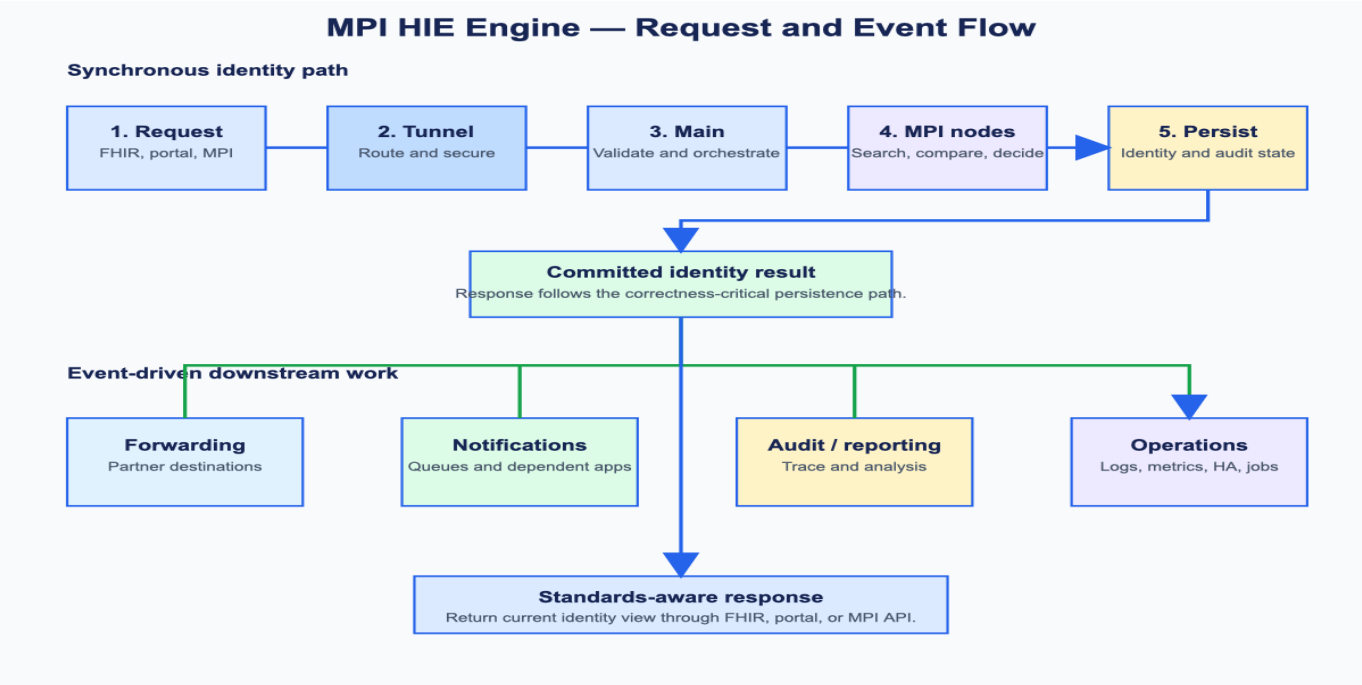
Topic introduction. An event-driven design moves downstream side effects out of the critical request path when they do not need to complete before the caller receives an identity result.

Why it matters. Forwarding, notifications, reporting, logging, and scheduled maintenance can consume significant time or fail independently. Decoupling them protects responsiveness and makes failures easier to isolate.

Internal working in this runtime. The MPI configuration invokes downstream handlers for notifications, forwarding, audit/reporting, local registration, and logging. The JAR also contains asynchronous loaders and asynchronous audit-search methods. This is asynchronous application processing, not proof of a non-blocking reactive JDBC stack.

Commit identity decision	Publish configured event	Forward / notify / audit / log	Retry and observe

Prepare next. Domain events, transactional outbox, queues, eventual consistency, at-least-once delivery, idempotent consumers, retries, dead-letter queues, correlation IDs, tracing, metrics.



6. Reactive programming: what to say accurately

Reactive programming is a concurrency model for composing asynchronous, non-blocking flows with demand control. It is useful when the entire path - HTTP client/server, database driver, messaging client, and application code - can avoid blocking threads.

It does not automatically make a system faster. For CPU-heavy matching it cannot reduce the work required. For blocking JDBC, simply wrapping calls in reactive types can move contention rather than remove it.

For this repository, the safe statement is: The core MPI performance path is based on indexing, distributed execution, bounded concurrency, buffered/batched JDBC persistence, and event decoupling. We found no evidence that the proprietary MPI JARs use Reactor Mono/Flux for that path.

Prepare next. Reactive Streams, Project Reactor, Mono and Flux, WebFlux, R2DBC, blocking versus non-blocking I/O, scheduler selection, backpressure, thread-per-request trade-offs, virtual threads.

Final preparation checklist

Explain the request path	**Explain one Java mechanism**	**Explain one DB mechanism**	**State benchmark boundary**

A strong presentation pairs each technical claim with a reason: indexes reduce candidate lookup, bounded executors control concurrency, batches reduce round trips, pools reuse expensive connections, and events keep non-critical work off the critical path.